Amortized Analysis

Motivation

- Many data structures, each with a set of operations defined on them
- Analysed time complexity
  - Worst case time complexity –worst case time for one execution of an operation
  - Eg. Insert in a heap, search in a BST
  - Average case time complexity –average time of one execution of an operation over all inputs
  - This is usually done by imposing a probability distribution on the inputs (usually uniform distribution, meaning all inputs are equally likely), and calculating the expected time of execution of the operation over all inputs

Motivation

- However, sometimes we are interested in considering a **set of operations together**
- The set may consist of **multiple executions of same or different operations**
- Trivial worst case complexity analysis will multiply the worst case time of each operation with the number of that operation and sum everything up
- This **may not be tight**: Operations may depend on each other in some ways to affect their time
- All executions of the same operation in the sequence **may not incur the worst case time**
- Question: Can we find the **average time of an operation** in a sequence of operations?

Amortized analysis

- Amortized analysis applies to certain algorithms that perform a sequence of similar operations
- Instead of bounding the cost of the sequence of operations by bounding the actual cost of each operation separately, an amortized analysis provides a worst-case bound on the actual cost of the entire sequence
- One advantage : although some operations might be expensive, many others might be cheap
- Can use amortized analysis when designing algorithms:
  - since the design of an algorithm and the analysis of its running time are often closely intertwined

Amortized analysis

- With amortized analysis, if you average over a sequence of operations :
  - then the average cost of an operation is small
  - even though a single operation within the sequence might be expensive
- Difference between Amortized analysis & average-case analysis
  - probability is not involved in amortized analysis
- An amortized analysis guarantees the *average performance of each operation in the worst case*

Three techniques: Amortized analysis

- Aggregate Analysis
- Accounting Method
- Potential Method

Aggregate Analysis

1. Determine an upper bound $T(n)$ on the total cost of a sequence of n operations

2. The average cost per operation = $T(n)/n$

3. Average cost is the amortized cost

4. Same amortized cost for all operations

Accounting Method

- Amortized cost for each operation
- If more than one type of operation, each type of operation may have a different amortized cost
- Overcharges some operations early in the sequence, storing the overcharge as “pre paid credit” on specific objects in the data structure
- Later, the credit pays for operations that are charged less than they actually cost

Potential Method

- Like accounting method:
  - Determine the amortized cost of each operation
  - may overcharge operations early on to compensate for undercharges later
- The potential method maintains the credit as the “potential energy "of the data structure as a whole
  - instead of associating the credit with individual objects within the data structure

Don’ts : coding

- The charges assigned during an amortized analysis are for analysis purposes only
- They need not and should not appear in the code
- Eg: If you assign a credit to an object x when using the accounting method
  - you have no need to assign an appropriate amount to some attribute, such as x: *credit* , in the code

Aggregate Analysis : Details

- For all n, a sequence of n operations takes T(n) *worst-case* time in total

- In the worst case, the average cost, or ***amortized cost***, per operation is therefore T (n)/n

- This amortized cost applies to each operation, even when there are several types of operations in the sequence

Aggregate Analysis : Example

- Stacks that have been augmented with a new operation
- Recall standard operations on stack:
  - PUSH(S, x) : pushes object x onto stack S
  - POP(S) pops the top of stack S and returns the popped object
  - Calling POP on an empty stack generates an error
  - Each of these operations runs in constant time ie O(1)
  - Cost of each operation is 1
- The total cost of a sequence of n PUSH and POP operations is therefore n
- Actual running time for n operations is therefore n

MULTIPOP (S, k)

- Removes the k top objects of stack S
- Popping the entire stack if the stack contains fewer than k objects
- Procedure assumes that k is positive
- otherwise, the MULTIPOP operation leaves the stack unchanged
- In the pseudocode for MULTIPOP, the operation STAC K-EMPTY returns TRUE if there are no objects currently on the stack and FALSE otherwise.

MULTIPOP (S, k) : Illustration

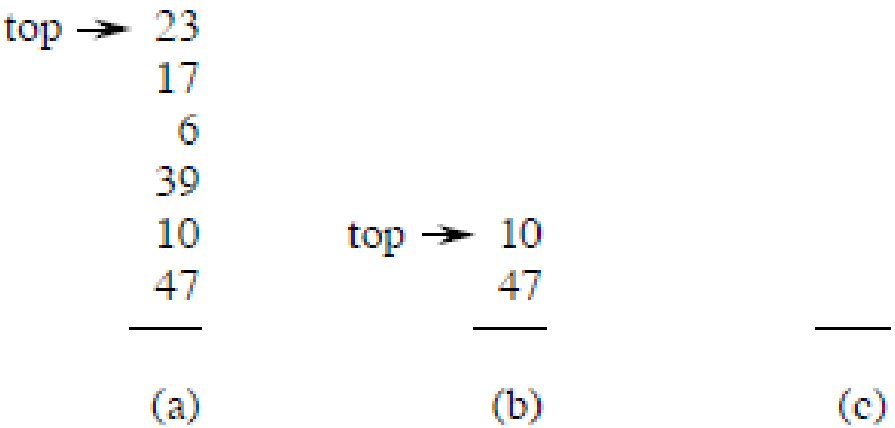

Figure 16.1 The action of MULTIPOP on a stack S, shown initially in (**a**). The top 4 objects are popped by MULTIPOP(S, 4), whose result is shown in (**b**). The next operation is MULTIPOP(S, 7), which empties the stack—shown in (**c**)—since fewer than 7 objects remained.

Pseudocode : MULTIPOP (S, k)

```
MULTIPOP(S,k)
While not STACK-EMPTY(S) and k > 0
        POP(S)
        k=k-1
```

What is the running time of MULTIPOP (S, k) on a stack of s objects?

Running time of MULTIPOP (S, k) on a stack of s objects

- Actual running time is linear in the number of POP operations actually executed
- The number of iterations of the **while** loop is the number min(s,k) of objects popped off the stack
- The total cost of MULTIPOP is min (s, k) and the actual running time is a linear function of this cost

Analysis: a sequence of n PUSH, POP, and MULTIPOP operations

- Stack is empty initially
- The worst-case cost of a MULTIPOP operation in the sequence is O(n)
  - since the stack size is at most n
- The worst-case time of any stack operation is therefore O(n)
- Hence a sequence of n operations costs $O(n^2)$
  - since the sequence contains at most n MULTIPOP operations costing O(n) each
- Although this analysis is correct,
  - the $O(n^2)$ result which came from considering the worst-case cost of each operation individually, is not tight

Aggregate Analysis

- A single MULTIPOP might be expensive
- As per aggregate analysis: any sequence of n PUSH, POP, and MULTIPOP operations on an initially empty stack has an upper bound on its cost of O(n) . Why?
  - An object cannot be popped from the stack unless it was first pushed
- # of POPs on a nonempty stack, **including calls within MULTIPOP** :
  - at most # of PUSH operations, which is at most n

Aggregate Analysis

- For any value of n, any sequence of n PUSH, POP, and MULTIPOP operations takes a total of O(n) time
- Averaging over the n operations gives an average cost per operation of O(n)/n = O(1)
- Aggregate analysis assigns the amortized cost of each operation to be the **average cost**
- In this example, therefore, all three stack operations have an amortized cost of **O(1)**

Recap : Aggregate Analysis

- Although the average cost, of a stack operation is O(1), the amortized analysis did not rely on probabilistic reasoning
- Instead, the amortized analysis yielded a *worst-case* bound of O(n)on a sequence of n operations
- Dividing this total cost by n yielded that the average cost per operation ie. the amortized cost is O(1)

Accounting Method : Details

- In the ***accounting method*** of amortized analysis:
  - you assign differing charges to different operations, with some operations charged more or less than they actually cost
- The amount that you charge an operation is its ***amortized cost***.
- When an operation's amortized cost exceeds its actual cost:
  - you assign the difference to specific objects in the data structure as ***credit***.
- Credit can help pay for later operations whose amortized cost is less than their actual cost
- Thus, amortized cost of an operation as being split between
  - its actual cost and credit (ie. either deposited or used up)
- Different operations may have different amortized costs
- This method differs from aggregate analysis:
  - in which all operations have the same amortized cost

Accounting Method : Details

- Choose the amortized costs of operations carefully
- If you want to use amortized costs to show that in the worst case the average cost per operation is small:
  - you must ensure that the total amortized cost of a sequence of operations provides an upper bound on the total actual cost of the sequence
- Moreover, the upper bound must apply to all sequences of operations

Accounting Method : Details

- Actual cost of the ith operation: c_i
- Amortized cost of the ith operation: $\hat{c}_i$
- For all sequence of operations:

$$\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$$

- The total credit stored in the data structure : difference between total amortized cost and total actual cost

$$\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i$$

- the total credit associated with the data structure must be nonnegative at all times

Accounting method : Details

- If you ever allowed the total credit to become negative (the result of undercharging early operations with the promise of repaying the account later on) :
  - then the total amortized costs incurred at that time would be below the total actual costs incurred
- For the sequence of operations up to that time, the total amortized cost would not be an upper bound on the total actual cost
- Thus, you must take care that the total credit in the data structure never becomes negative

Example : Stack

- Actual costs of the operations were:
  - PUSH -1
  - POP -1
  - MULTIPOP- min(s,k)
- Amortized costs :
  - PUSH -2
  - POP-0
  - MULTIPOP – 0
- The amortized cost of MULTIPOP is a constant (0), whereas the actual cost is variable,
  - thus all three amortized costs are constant

How to pay for sequence of operations ?

- Charge the amortized costs to pay for the sequence of operations
- Let $1 represent each unit of cost
- Initially the stack is empty
- Assuming a stack of plates in this scenraio
- PUSH a plate on to the stack
  - Use $1 to pay the actual cost of the PUSH
  - Leaving $1 leaving a credit of $1 (out of the $2 charged)
- Place that $1 of credit on top of the plate
- At any point in time, every plate on the stack has $1 of credit on it

How to pay for sequence of operations ?

- The $1 stored on the plate serves to
  - prepay the cost of popping the plate from the stack
- A POP operation incurs no charge:
  - pay the actual cost of popping a plate by taking the $1 of credit off the plate
- Thus, by charging the PUSH operation a little bit more,
  - we can view the POP operation as free

MULTIPOP

- MULTIPOP operation also incurs no charge,
  - since it's just repeated POP operations, each of which is free
- If a MULTIPOP operation pops k plates,
  - then the actual cost is paid by the k dollars stored on the k plates
- Because each plate on the stack has $1 of credit on it, and the stack always has a nonnegative number of plates,
  - the amount of credit is always nonnegative
- Thus, for *any* sequence of n PUSH, POP, and MULTIPOP operations,
  - the total amortized cost is an upper bound on the total actual cost
- Since the total amortized cost is O(n), so is the total actual cost

Potential Method

- Instead of representing prepaid work as credit stored with specific objects in the data structure,
  - the ***potential method*** of amortized analysis represents the prepaid work as potential energy / potential
  - which can be released to pay for future operations
- The potential applies to the data structure as a whole rather than to specific objects within the data structure

Working : Potential method

- Starting with an initial data structure D_0
  - a sequence of n operations occurs
- For each i, $D_1, D_2, \ldots, D_n$:
  - c_i be the actual cost of the ith operation
  - D_i be the data structure that results after applying the ith operation to data structure D_{i-1}
- A ***potential function*** ϕ maps
  - each data structure D_i to a real number, $\phi(D_i)$ which is the ***potential*** associated with D_i

Amortized cost

- The ***amortized cost*** $\hat{c}_i$
  - of the ith operation with respect to potential function ϕ is defined by

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

- The amortized cost of each operation is therefore its actual cost plus the change in potential due to the operation

Amortized cost

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1}))$$

$$= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0) .$$

- The second line/ term results, because of the telescoping series
- If we can define a potential function ϕ such that $\phi(D_n) >= \phi(D_0)$, then total amortized cost gives an upper bound on the total actual cost

Potential method

- In practice, you don't always know how many operations might be performed
- If you require that $\phi(D_i) >= \phi(D_0)$ for all i, then
  - There is a guarantee that we have paid in advance
- Define $\phi(D_0)$ to be 0, show that $\phi(D_i) >= 0$ for all i
- Intuitively, if the potential difference $\phi(D_i) - \phi(D_{i-1})$ of all the ith operation :
  - is positive, then the amortized cost represents an overcharge to the ith operation, and the potential of the data structure increases
  - is negative, then the amortized cost represents an undercharge to the ith operation, and the decrease in the potential pays for the actual cost of the operation

Potential functions

- The amortized costs depend on the choice of the potential function
- Different potential functions may yield different amortized costs
- But amortized costs are upper bounds on the actual costs
- Tradeoffs in choosing a potential function
- The best potential function to use depends on the desired time bounds

Example: Stack

- Potential function ϕ on a stack:
  - number of objects on the stack
- The potential of the empty initial stack, $\phi(D_0)=0$
- Since the number of objects in the stack is never negative,
  - the stack D_i that results after the ith operation has nonnegative potential ie $\phi(D_i) >= 0$

Computation of amortized costs of stack operations

- If the ith operation on a stack containing s objects is a PUSH operation, then the potential difference is: $\phi(D_i) - \phi(D_{i-1}) = (s+1)-s = 1$
- Amortized cost as per the definition:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

- $\hat{c}_i = 1+1 = 2$

Computation of amortized costs of stack operations

- Suppose that the ith operation on the stack of s objects is MULTIPOP(S,k)
  - which pops off min (s,k) elements from the stack
- The actual cost of the operation is k ’ , and the potential difference is

$$\Phi(D_i) - \Phi(D_{i-1}) = -k' .$$

Computation of amortized costs of stack operations

- The amortized cost of the MULTIPOP operation:

$$\begin{aligned}\widehat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= k' - k' \\ &= 0 .\end{aligned}$$

- The amortized cost of an ordinary POP operation is 0.

Computation of amortized costs of stack operations

- The amortized cost of each of the three operations is O(1)
  - Thus the total amortized cost of a sequence of n operations is O(n)
- Since, $\phi(D_i) >= \phi(D_0)$,
  - the total amortized cost of n operations is an upper bound on the total actual cost
- The worst-case cost of n operations is therefore O(n)

Another example : Amortized analysis k-bit binary counter

- Implement k-bit binary counter
  - That counts upward from 0
- An array A[0:k-1] of bits represents the counter
- A binary number x that is stored in the counter has its lowest order bit in A[0] and its highest-order bit in A[k-1], so that x is :

$$x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

- Initially x=0, and thus A[i]=0 for i=0 to k-1

Increment the binary counter

- To add 1 (modulo 2^k) to the value in the counter, call the INCREMENT procedure :

```
INCREMENT(A, k)
i = 0
while i < k and A[i] == 1
    A[i] = 0
    i = i + 1
if i < k
    A[i] = 1
```

8-bit binary counter

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Figure 16.2 An 8-bit binary counter as its value goes from 0 to 16 by a sequence of 16 INCREMENT operations. Bits that flip to achieve the next value are shaded in blue. The running cost for flipping bits is shown at the right. The total cost is always less than twice the total number of INCREMENT operations.

Reading exercise

- Amortized analysis of implementing a k-bit binary counter
- Reference : CLRS 4thedition

Reference

Introduction to Algorithms, Fourth Edition

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein